

Project 1: Double The Priority

Instructor: Sadab Hafiz

Due: June 20, 2026



Introduction

The heap data structure is incredibly powerful, offering $O(1)$ access to the min/max item while maintaining $O(\log(n))$ time complexity for standard insertions and deletions. In this project, you will implement a flexible `Heap` class capable of acting as either a min-heap or a max-heap, utilizing internal index tracking to map item locations across structures efficiently. You will then leverage this implementation to construct a synchronized, double-ended priority queue (`DEPriorityQueue`) that supports access to both high and low priority items in $O(1)$. This assignment will heavily test your mastery of core C++ principles, including the Rule of Five, smart pointers, and class templates.

Beyond the data structure design, this project serves as a practical introduction to a professional development workflow. You will verify your local C++ environment, practice code documentation, and submit your code to Gradescope via GitHub. Mastering these fundamental skills now will allow you to focus on advanced coding challenges in future assignments. Gaining a deep understanding of what each line of your code does is excellent preparation for the midterm. This will be a good learning experience!

Setting up GitHub

This step is necessary to get started with the project. You will use a personal, private Git repository to manage your codebase and track your progress. To get started, visit the [GitHub website](#) and sign up for a free account if you do not have one already. After creating your account, I highly encourage you to watch [this video by Brian Yu](#) which covers the basics of Git and GitHub.

Repository Setup Instructions:

1. Create a **new repository** on your GitHub account. Name it CSCI335-DEPriorityQueue.
2. **IMPORTANT:** You must set your repository visibility to **Private**. Making your repository public will expose your solution to other students, which is a **violation of the academic integrity** policy.
3. Download the project starter code from **Brightspace**, initialize a git repository locally (`git init`), link it to your GitHub private remote, and push your initial commit.

When it is time to submit, log into Gradescope, select this assignment, and choose the option to submit via GitHub. Connect your GitHub account and select your private repository. **Do not create forks of any public tracking repositories** to ensure your development history remains private. You can find submission instructions [here](#). Make sure to refer back to these instructions when it's time to submit.

Documentation and Design

Documentation and design together are worth 15% of each project grade. The requirements are:

Documentation Requirements (10%)

- All files must begin with a top-level block comment containing at least your name, the date, and a brief description summarizing the purpose of the code implemented in that file.
- All functions in the `.hpp` files must include a structured Doxygen-style block comment:

```
/**
 * @brief Describes what the function does in general terms.
 *
 * @pre Describes any preconditions that must be true before calling.
 * @param Description of an individual parameter (repeat for each parameter).
 * @return Description of the returned value and type.
 * @post Describes postconditions or side-effects after execution.
 * @throws Describes any exceptions the function may throw.
 */
```

For the required project functions, these blocks are provided in your starter instructions. However, you are strictly required to write them for any additional helper functions you choose to implement.

- While header documentation describes interface usage, you must use inline comments (`//`) inside your `.cpp` files to clarify complex implementation details (such as loops, intricate logic, or conditionals):

```
// Trace through the tracking array to find the target item index
for (DataType x : some_arr_) {
    // If a priority match is found, synchronize the secondary heap
    if (...condition...) {
        ...implementation code...
```

```

    }
    else {
        ...implementation code...
    }
}

```

If a block of code is not immediately obvious to an outside reader, leave a brief inline comment. Do not comment every single line. Self-explanatory code should remain uncommented.

Design Requirements (5%)

- You should follow the naming convention below:

```

string my_variable;           // Snake_case for local variables
class MyClass {};            // PascalCase for class names
MyClass class_instance;      // Snake_case for instances
string my_data_member_;      // Trailing underscore for private members
void myFunction();           // camelCase for member functions
const int MY_CONSTANT = 10;  // ALL_CAPS for constants

```

- Maintain architectural consistency throughout your codebase. This applies to your choice of include guards (`#pragma once` vs. `#ifndef`), member ordering inside classes, and general formatting choices.
- Adhere strictly to Object-Oriented Programming (OOP) principles. Public data members and global variables are prohibited unless explicitly authorized by the project specification.
- Use the `const` keyword and pass-by-reference (`&`) design patterns appropriately. Functions that do not modify private class data members must be marked `const`.
- Do not include `using namespace std;` in any header (`.hpp`) files or any files in case of class templates.
- **Resource and Memory Management:** Ensure all dynamically allocated blocks are explicitly freed or cleanly managed using appropriate smart pointers. If a class overrides a custom destructor, ensure the remaining Rule of Five operators are correctly handled.

Helpful Tips

If you are developing in VS Code, I highly recommend utilizing extensions to automate your documentation generation. The starter frameworks provided use templates compatible with [Doxygen](#), a tool standard for auto-generating C++ documentation. Consider installing the following extensions:

- [Doxygen Documentation Generator](#) : Automatically generates structured comment block skeletons.
- [C/C++ for VSCode](#) : Official extension providing IntelliSense autocomplete, syntax formatting, etc.

If you lack a working local C++ environment, reach out immediately. The provided starter header templates serve as a direct visual baseline for your documentation requirements.

Starter Code Tips

In the provided files, make sure you address each `TODO` comment and delete the comments after addressing them. If you plan to modify a function that is explicitly stated not to be modified, make a copy of the original. You can also look at the original starter files from GitHub commit history.

Task A: Heap Class



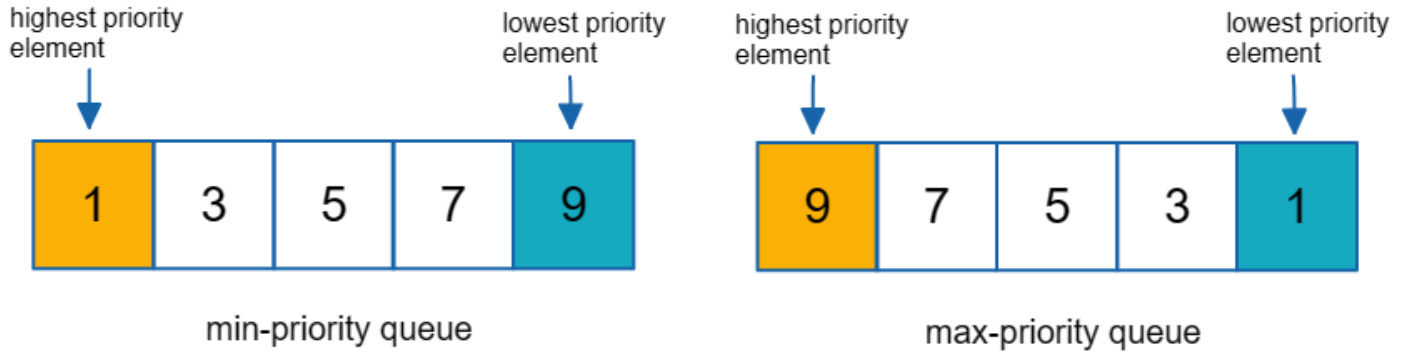
Your first task is to implement the templated `Heap` class. The header file is available in your repository. Carefully go through each of the function comments to figure out how to implement them. Some functions can be used to implement others. I encourage you to read the entire header before writing code. To assist you, the `Heap.cpp` starter file includes a fully implemented `removeAt(size_t idx)` function. **Do not modify this function.** Study it closely as it demonstrates how the helper functions interact and reveals how the heap tracks its items using `minHeapIndex` and `maxHeapIndex`.

Tips for implementation:

- **prioritize:** This function can reduce the need to write redundant if-statements to check the heap type every time an operation differs in functionality in a min-heap compared to a max-heap. If `isMinHeap_` is true, it should return true if node `a` has a strictly lower priority value than node `b`. If false, it evaluates based on higher priority value.
- **Index Tracking:** The double-ended priority queue in Task B relies entirely on nodes knowing exactly where they are inside both heaps. Whenever a node changes position in the underlying `std::vector` (during a swap in `bubbleUp`, `bubbleDown`, or a push), you **must** update its corresponding `minHeapIndex` or `maxHeapIndex` to match its new vector index.
- **Deep Copies:** You must allocate brand-new `HeapNode` structures in memory rather than copying the shared pointers themselves.

A `main.cpp` file is provided to demonstrate how the `Heap` class can be used to perform a `HeapSort`. This file is very basic and does not test edge cases (such as popping from an empty heap or deep copying). You are expected to modify this file or create your own tests to thoroughly verify your implementation. **You are NOT allowed to use any external library implementations** (`std::make_heap`, `std::priority_queue`, etc.) **for this task.**

Task B: Double-Ended Priority Queue



Now that you have implemented a fully functional `Heap` class that tracks indices for both max-heap and min-heap, you are ready to use it to implement the `DEPriorityQueue`. While a standard priority queue only allows access to either the minimum or maximum item, a double-ended priority queue provides $O(1)$ access and $O(\log(n))$ deletion for both the minimum and maximum items simultaneously. You will achieve this by using two internal heaps within your `DEPriorityQueue` class: a `minHeap_` and a `maxHeap_`. The main challenge will be ensuring that these two heaps stay perfectly synchronized at all times.

Synchronizing The Heaps:

Every item exists in both heaps simultaneously. A single data item and its priority are packed into a single `HeapNode`, which is then managed by a shared pointer (`std::shared_ptr<HeapNode<T>>`) across both heaps. This should be reflected when you implement the following:

- **push:** When a new item is added, **you must allocate exactly one `HeapNode`**. You will then push this same shared node pointer into both `minHeap_` and `maxHeap_`. Your fully functional `Heap` class should handle updates to the indices as the items are moved around to preserve the heap property.
- **popMin and popMax:** When popping the minimum item, the item is easily identified at the root of `minHeap_`. However, it still exists somewhere inside `maxHeap_` and you need to deal with it. Before you remove the node from `minHeap_`, you should track its `maxHeapIndex`. You will use this index to explicitly remove the node from the other heap using `maxHeap_.removeAt(idx)`. The same symmetric logic applies when you are implementing `popMax`.

Tips for Implementation:

- **Move:** You must implement the Move Constructor and Move Assignment Operator yourself without using `= default`. Ensure that you cleanly transfer ownership of internal resources and leave the other queue in a valid, empty state.
- **Copy:** Since the `Heap` class doesn't provide a way to iterate through its internal vector, you can complete a deep copy using the public interface:
 - Create a temporary copy of one of the heaps from the parameter `DEPriorityQueue` object using the `Heap` copy assignment operator implemented in Task A.
 - Successively inspect items from this temporary heap using `peek()` and remove them via `pop()`.
 - Feed the extracted item's data and priority directly into your local queue's `push()` function.
 - This triggers new node allocations internally, and handles dynamic memory as expected.
- **Heap Operations:** In general, use the already implemented heap operations where applicable.

Compiling and Testing

Since `DEPriorityQueue` requires `Heap` to be implemented correctly, I recommend implementing and testing the `Heap` class first before moving on to `DEPriorityQueue`. Assuming you implement everything, you can compile everything with `g++ --std=c++11 main.cpp`. Note that we do not compile `Heap.cpp` or `DEPriorityQueue.cpp` as they are class templates. Including them in `main.cpp` will cause them to be compiled automatically. You should be used to compiling and running your code locally.

Make sure to test all functions and edge cases by writing tests to ensure they work as specified. Testing is a very important skill that you need to learn by trial and error. If you get an error, look at the error message and find out which line it is coming from. I recommend implementing each function and testing it before implementing the next one.

Submissions

You will submit your code to Gradescope directly from your private GitHub repository. Refer back to the **Setting up GitHub** section of this document to review the instructions and videos if you haven't done so already. The autograder will grade: `Heap.hpp`, `Heap.cpp`, `DEPriorityQueue.hpp`, and `DEPriorityQueue.cpp`. Although Gradescope allows multiple submissions, it is not a place to test your code every time you make changes. You have to test it locally before uploading to Gradescope. To prevent the abuse of the autograder and to encourage testing, **the autograder feedback will be limited**. Your code needs to compile on Gradescope in order to get any feedback from the autograder. Furthermore, the autograder will be released near the due date which incentivizes testing your own code instead of relying on the autograder. Once you pass the test cases on Gradescope, make sure to verify the documentation and design requirements for full credit.

Plagiarism and Citing

Your code in all the `.cpp` files should not 100% match with another student. If that happens, I will assume it is a violation of academic integrity. It can happen with the use of generative AI, it can also happen if you both copy code from the same source. If you use libraries or functions that are not mentioned in the slides, or I didn't teach in class, you have to cite them with an inline comment. You should also be able to explain such lines of code if I ask you to explain what you wrote. In general, if you are not sure if something is appropriate for the project, you can reach out to confirm. Any proven academic dishonesty will result in a significant penalty to your assignment grade depending on the violation.

Additional Tips

Here are more tips to help you. If more clarification is needed, they will be announced on **Brightspace**.

- When a node is first introduced into a heap's vector container using `push()`, its tracking index must be immediately mapped to its current vector index *before* any bubbling operations occur. A node must always know its location, even if it never leaves its initial entry position.
- Whenever a node is popped from a heap using the `pop()` function, you should reset the tracked indices of the removed node to -1 to pass the autograder tests.